Entity-Relationship Diagram

Project Title	AI Academic Mentor
Team	CSE4104-7D-T06
Section	7D
Institution	Northern University of Business & Technology Khulna (NUBTK)
Course	CSE 4104 — Software Development III
Document	ER Diagram
Date	16 November 2026
GitHub Repository	https://github.com/Santu-Mondal/AI-Academic-Mentor-Personalized-Learning-Study-Assistant/tree/411142fcd5c145a92f334e7a9af446e864ef6c69/documentation
Instructor	Md. Riaz Mahmud, Assistant Professor, CSE — NUBTK

Team Information

SL No.	Member Name	Student ID	Role
1	Efaz Rahman	11230121162	Team Leader
2	Santu Mondal	11230321312	Frontend Developer
3	MD Ruhan Tanveer	11230321335	Backend Developer AI Integration Lead

1. Database Entity-Relationship Diagram

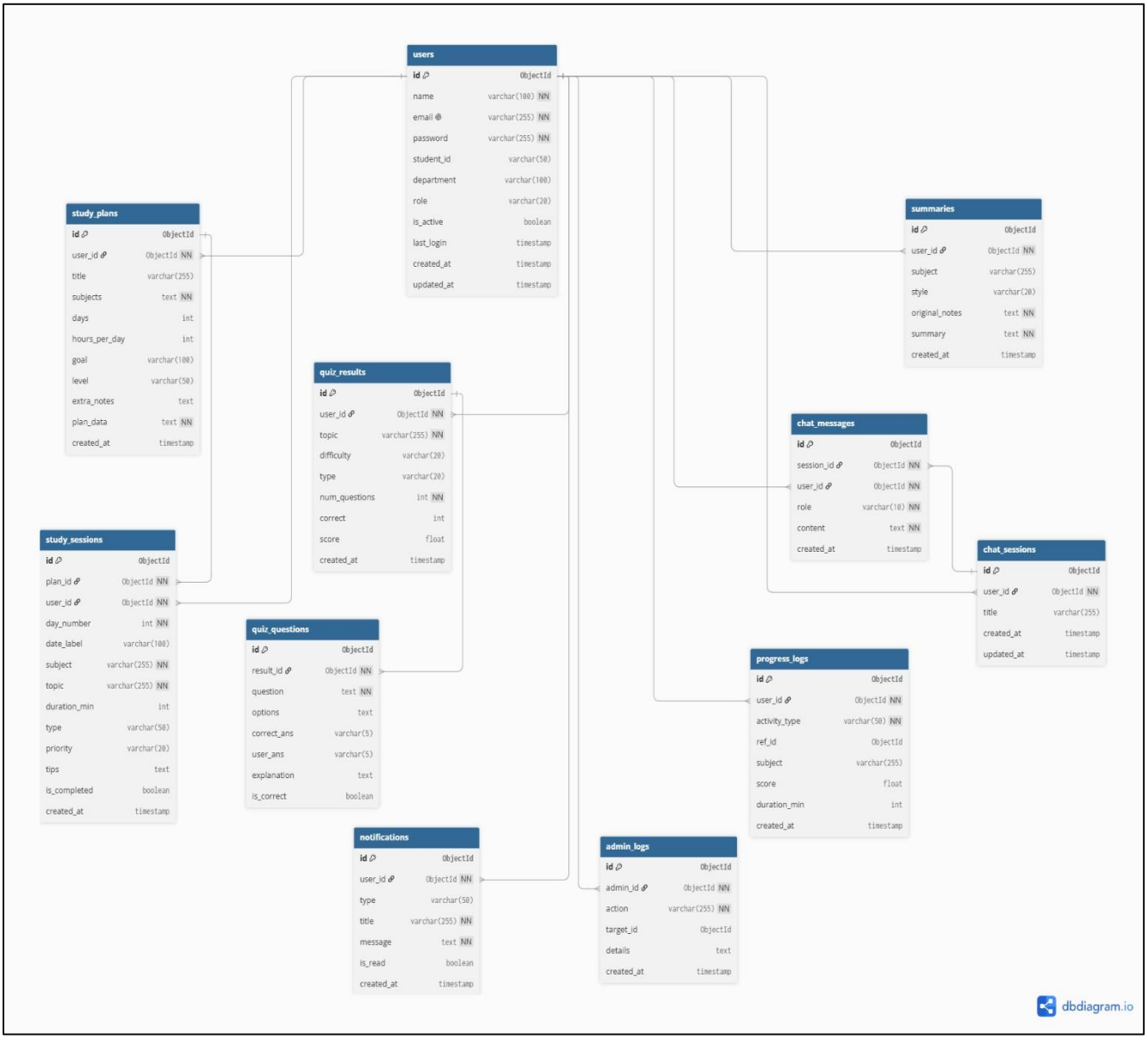


Fig — AI Academic Mentor Database Entity-Relationship Diagram

2. ER DIAGRAM — DETAILED DESCRIPTION

2.1 Overview

The Entity-Relationship (ER) diagram for the AI Academic Mentor system models the complete relational structure of the application's MongoDB-backed database. Designed using dbdiagram.io, the schema comprises twelve interconnected entities (collections) that collectively govern user identity, AI-powered feature data, learning progress, administrative oversight, and real-time communication. Every primary entity uses Object_Id as its primary key, reflecting MongoDB's native document-identifier convention. Relationships are expressed through Object_Id foreign-key references, consistently denoted by the key icon and the NN (Not Null) constraint where referential integrity is mandatory.

2.2 Central Entity: users

The users table is the nucleus of the entire schema. It stores foundational identity attributes — name, email (unique), password, student_id, department, and role — alongside operational flags such as is_active (boolean) and last_login (timestamp). All eleven remaining entities reference users.id as a foreign key (user_id), establishing a star-topology relationship pattern. This design centralizes authentication and authorization: a single users record governs access to study planning, quizzes, chat, summarization, notifications, and progress tracking. The role field (varchar 20) facilitates role-based access control, differentiating between regular students and administrative users.

2.3 Study Planning Cluster: study_plans & study_sessions

The study_plans entity persists AI-generated personalized study schedules. Each plan belongs to a user (user_id → users.id) and captures high-level planning metadata: title, subjects (text), total days, hours_per_day, goal, difficulty level, extra_notes, and the full AI-generated plan_data stored as a TEXT blob. The study_sessions entity operationalizes these plans at the session level. Each session references both the parent plan (plan_id → study_plans.id) and the owning user (user_id), capturing granular attributes such as day_number, date_label, subject, topic, duration_min, session type, priority, tips, and a boolean is_completed flag. This two-tier structure separates high-level plan configuration from day-by-day session execution tracking, enabling progress visualization and adaptive re-planning.

2.4 Quiz & Assessment Cluster: quiz_results & quiz_questions

The quiz_results entity records each quiz attempt at the aggregate level — topic, difficulty, type (MCQ / True-False), total num_questions, correct answers count, and a float score. quiz_questions holds the granular per-question data for each quiz result, linked via result_id (→ quiz_results.id). Each question record contains the question text, options (JSON/text), correct_ans, the student's user_ans, an explanation, and a boolean is_correct. This parent-child design supports detailed per-question analytics and review, allowing the frontend to render both high-level score summaries and question-by-question explanations — a key affordance for adaptive learning feedback.

2.5 Conversational AI Cluster: chat_sessions & chat_messages

The chat_sessions entity represents a named multi-turn conversation session, owned by a user and identified by a title and updated_at timestamp to support session resume and history listing. The chat_messages entity stores individual messages within a session, referenced by session_id (→ chat_sessions.id) and user_id. The role field (varchar 10, NN) distinguishes user-authored messages from AI-generated assistant responses, mirroring the OpenAI / Gemini API message role convention. The content (TEXT, NN) field holds the message body. This structure enables the backend to reconstruct full conversation context for multi-turn Gemini API calls, maintaining coherent academic mentoring interactions.

2.6 Content Summarization: summaries

The summaries entity persists AI-generated document and note summaries. Each record links to a user (user_id) and captures the subject, summarization style (e.g., concise, detailed, bullet-point — varchar 20), the original_notes submitted by the student (TEXT, NN), and the AI-produced summary (TEXT, NN). The style field enables the Summary Service to select the appropriate Gemini prompt template, delivering context-aware condensation tailored to student preferences. Timestamped creation allows students to maintain a personal library of past summaries.

2.7 Progress Tracking: progress_logs

The progress_logs entity provides a universal activity ledger for all user interactions across the platform's AI features. Each log entry records the user_id, activity_type (varchar 50, NN — e.g., 'quiz_completed', 'summary_generated', 'study_session_done'), a polymorphic ref_id (Object_Id) pointing to the relevant entity instance, a subject label, an optional float score, and duration_min. This polymorphic reference pattern avoids separate log tables per feature and enables a unified progress dashboard. The data stored here is the foundation for generating learning analytics, streak tracking, and subject-level performance reports.

2.8 Notification System: notifications

The notifications entity supports in-app alert delivery for study reminders, quiz results, and system messages. Each notification belongs to a user (user_id, NN) and carries a type (varchar 50), a title (varchar 255, NN), a message body (TEXT, NN), and an is_read boolean for read-state management. This schema supports both push-style notifications triggered by backend events (e.g., study session due, quiz feedback ready) and system announcements added in Week 4 of the project roadmap, which the current architecture document notes as pending inclusion in the next revision.

2.9 Administrative Audit: admin_logs

The admin_logs entity implements a full audit trail for administrative actions. It captures the admin_id (→ users.id) performing the action, a descriptive action string (varchar 255, NN), a polymorphic target_id (Object_Id, nullable) identifying the affected resource, and a free-text details field for contextual notes. This design supports GDPR-aligned audit requirements, enabling administrators to trace any create, update, or delete operation performed on the platform's data. The separation of admin_logs from progress_logs maintains a clean boundary between student-facing analytics and system-level governance.

2.10 Schema Design Observations & Architectural Alignment

Several design choices merit specific attention. First, all timestamp fields use a consistent created_at convention, with updated_at added where mutable state requires change detection (users, chat_sessions). Second, the TEXT type is used liberally for variable-length AI-generated content (plan_data, summary, content, options, explanation), avoiding premature length constraints on machine-generated output. Third, the polymorphic ref_id in progress_logs and target_id in admin_logs avoids over-normalization at the cost of enforced referential integrity, a deliberate trade-off suited to MongoDB's flexible document model. Fourth, the schema structurally mirrors the five service modules described in the system architecture: the Quiz Service maps to quiz_results and quiz_questions; the Study Plan Service to study_plans and study_sessions; the Chat Service to chat_sessions and chat_messages; the Summary Service to summaries; and the Authentication Service to users. This one-to-one alignment between service layer and data model ensures high cohesion and low coupling, facilitating independent development and testing of each functional domain.